HW 4

Sam Ly

February 26, 2026

Final code:
https://github.com/samlyme/Assignments/tree/main

Chapter 8

The code for the challenge questions can be found here:
https://github.com/samlyme/Assignments/tree/cs4080/ch8/challenge

1. The REPL no longer supports entering a single expression and automatically printing its result value. That's a drag. Add support to the REPL to let users type in both statements and expressions. If they enter a statement, execute it. If they enter an expression, evaluate it and display the result value.

   This can be done by performing a simple check for semicolons in the provided token list. If there are no semicolons, then we treat it as an expression.

```
private static void run(String source) {
    Scanner scanner = new Scanner(source);
    List<Token> tokens = scanner.scanTokens();
    Parser parser = new Parser(tokens);

    boolean hasSemicolon = false;
    for (Token token : tokens) {
        if (token.type == TokenType.SEMICOLON) {
            hasSemicolon = true;
            break;
        }
    }

    if (hasSemicolon) {
        List<Stmt> statements = parser.parse();

        if (hadError) return;

        interpreter.interpret(statements);
    } else {
        Expr expr = parser.parseExpression();
        System.out.println(interpreter.evaluate(expr));
    }
}
```

2. Maybe you want Lox to be a little more explicit about variable initialization. Instead of implicitly initializing variables to nil, make it a runtime error to access a variable that has not been initialized or assigned to, as in:

   This can be done by adding a HashSet that tracks which variables have been initialized.

```
package com.mylox.lox;

import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

class Environment {
    final Environment enclosing;
    private final Map<String, Object> values = new HashMap<>();
    private final Set<String> initialized = new HashSet<>();

    Environment() {
        this.enclosing = null;
    }

    Environment(Environment enclosing) {
        this.enclosing = enclosing;
    }

    void define(String name, Object value) {
        values.put(name, value);
        if (value != null) initialized.add(name);
    }

    Object get(Token name) {
        if (values.containsKey(name.lexeme)) {
            if (!initialized.contains(name.lexeme)) {
                throw new RuntimeError(name, "Variable not initialized.");
            }
            return values.get(name.lexeme);
        }

        if (enclosing != null) return enclosing.get(name);

        throw new RuntimeError(name,
                "Undefined variable '" + name.lexeme + "'.");
    }

    void assign(Token name, Object value) {
        if (values.containsKey(name.lexeme)) {
            values.put(name.lexeme, value);
            initialized.add(name.lexeme);
            return;
        }

        if (enclosing != null) {
            enclosing.assign(name, value);
            return;
        }

        throw new RuntimeError(name, "Undefined variable '" + name.lexeme + "'.");
    }
}
```

3. What does the following program do?

```
var a = 1;
{
  var a = a + 2;
  print a;
}
```

What did you expect it to do? Is it what you think it should do? What does analogous code in other languages you are familiar with do? What do you think users will expect this to do?

This creates a block scope that redeclares the variable a as 3. When the block scope exits, a is still 1. This behaves how I would expect it to. This behavior is also useful as it lets the user redefine variables with the same name in scopes without causing strange mutations outside. This is analogous to how JS handles scopes.

Chapter 9

The code for the challenge questions can be found here:
https://github.com/samlyme/Assignments/tree/cs4080/ch9/challenge

1. A few chapters from now, when Lox supports first-class functions and dynamic dispatch, we technically won't need branching statements built into the language. Show how conditional execution can be implemented in terms of those. Name a language that uses this technique for its control flow.

   To be clear, branching statements will be needed for the actual implementation, but at the language level (user-facing), we can use a "boolean class". In fact, at the very low level, dynamic dispatch is still just a branch or jump statement.

   The boolean class will have a method like `cond`, or in the case of SmallTalk, `thenElse`. This method takes in two functions/closures, `then` and `else`. The boolean class will also have two sub-classes, a 'Truthy' and a 'Falsy'.

   Objects of the 'Truthy' class will always execute the `then` function when the `thenElse` method is called. Conversely, objects of the 'Falsy' class will always execute the `else` function.

   When we instantiate an object of the boolean class, the implementation/vm actually returns an instance of either 'Truthy' or 'Falsy'. Remember, this is how the language "hides" the branching logic. In a sense, we turn tradidtional boolean operations into a factory function that returns instances of the boolean class.

2. Likewise, looping can be implemented using those same tools, provided our interpreter supports an important optimization. What is it, and why is it necessary? Name a language that uses this technique for iteration.

   To implement `while` loops without branching statements, we can use a similar trick, but now we translate the looping logic into a recursive call. This is because without branching, we must use another dynamic dispatch to end the loop.

   So, we would implement a `while` method that takes in two functions, a `stmt` and `nextCond`, which returns a boolean object. The `while` method will execute `stmt`, and then evaluate `nextCond`. Then, we will call into the value returned by `nextCond.while`. If `nextCond` returns a Falsy object, we are done with the loop.

   One issue that can arise is stack usage because each recursive call requires a new stack frame when implemented naively. However, we can fix this by using Tail Recursion. When we make the recursive call as the very last statement in a stack frame, we can actually overwrite the stack frame in place, since we know we do not need the current frame's data anymore. This way, we reduce our memory complexity from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ where n is the number of interations.